

Replication Plan

Integer Sequences from Configurations in the Hausdorff Metric Geometry via Edge
Covers of Bipartite Graphs

OSTI ID: 1997354

Auto-generated Replication Blueprint

April 8, 2026

Contents

1	Paper Overview	2
2	Detailed Workflow	2
2.1	Phase 1: Mathematical Framework	2
2.2	Phase 2: Bipartite Graph Construction	2
2.3	Phase 3: Integer Sequence Generation	2
2.4	Phase 4: Closed-Form Verification	2
2.5	Phase 5: Validation	3
3	Datasets	3
4	Models, Tools, and Software	3
4.1	Hardware Requirements	3

Paper Overview

This paper derives integer sequences arising from counting configurations in finite metric spaces under the Hausdorff metric. The key mathematical objects are bipartite graphs connecting points of two finite configurations, where edges represent distance relationships. The paper enumerates edge covers of these bipartite graphs to count the number of distinct Hausdorff distance configurations. All computations are purely combinatorial/algebraic and produce verifiable integer sequences (many registered in the OEIS).

Detailed Workflow

Phase 1: Mathematical Framework

1.1 Implement Hausdorff distance computation.

$$d_H(A, B) = \max \left(\max_{a \in A} \min_{b \in B} d(a, b), \max_{b \in B} \min_{a \in A} d(a, b) \right)$$

1.2 Enumerate finite configurations.

- For a discrete metric space $X = \{1, 2, \dots, n\}$, enumerate all non-empty subsets $A, B \subseteq X$.
- Compute $d_H(A, B)$ for all pairs.

Phase 2: Bipartite Graph Construction

2.1 Construct the bipartite graph $G(A, B)$.

- Vertices: elements of A on one side, elements of B on the other.
- Edges: connect $a \in A$ to $b \in B$ if $d(a, b) \leq r$ for a given radius r .

2.2 Enumerate edge covers.

- An edge cover is a set of edges such that every vertex is incident to at least one edge.
- Count edge covers using inclusion-exclusion or the permanent-like formula from the paper.

Phase 3: Integer Sequence Generation

3.1 Systematically vary parameters (n , $|A|$, $|B|$, distance threshold).

3.2 Compute the counting sequences for each parameter combination.

3.3 Tabulate results in the format matching the paper's tables.

3.4 Cross-reference with OEIS to verify known sequences.

Phase 4: Closed-Form Verification

4.1 Verify closed-form expressions given in the paper against brute-force enumeration.

4.2 Verify recurrence relations for generating larger terms.

4.3 Test generating functions where provided.

Phase 5: Validation

5.1 Compare all generated sequences term-by-term with the paper’s stated values.

5.2 Verify OEIS entries match (by A-number).

5.3 Extend sequences beyond what’s in the paper as an additional check.

Datasets

Dataset / Source	Type	Location / Notes
OEIS (Online Encyclopedia of Integer Sequences)	Reference integer sequences	https://oeis.org/
Paper’s tables of sequences	Reference	Extracted from the paper
All other data	Synthetic (enumeration)	Generated by the algorithms

Models, Tools, and Software

Tool / Library	Version	Purpose / URL
Python	≥ 3.8	Primary implementation language
SageMath	≥ 9.5	Combinatorial enumeration, graph theory https://www.sagemath.org/
NetworkX	≥ 2.6	Bipartite graph construction and analysis https://networkx.org/
SymPy	≥ 1.10	Symbolic computation, generating functions https://www.sympy.org/
itertools (stdlib)	built-in	Combinatorial enumeration of subsets
NumPy	≥ 1.21	Array operations
Matplotlib	≥ 3.4	Visualization of graph structures

Hardware Requirements

- Any modern laptop. Enumeration is exponential in n but the paper works with small n .
- For extending sequences to larger n : more RAM and multi-core CPU helpful.